

Campus Share Development Bible

Fast-track engineering guide for a one-day college project using React, Vite, Firebase and Tailwind.
This document is intentionally practical.

Page 2: Project Overview

Campus Share is a peer-to-peer item rental platform for students inside a college or hostel. The objective is to demonstrate CRUD, authentication and cloud storage.

Page 3: Problem Statement

Students often need calculators, lab coats, books or sports equipment for short periods.

Page 4: Goals

Deliver a working MVP in one day. Prioritize reliability over features.

Page 5: Scope

Login, browse listings, add listing, search, item details and contact owner.

Page 6: Out of Scope

Payments, chat, ratings, notifications, admin dashboard, maps and AI features.

Page 7: Tech Stack

React + Vite, Tailwind CSS, Firebase Authentication, Firestore, Firebase Storage.

Page 8: Folder Structure

Describe src/components, pages, firebase, services, hooks, assets.

Page 9: Routing

/login, /home, /add, /item/:id.

Page 10: Authentication

Google Sign-In using Firebase Authentication.

Page 11: Firestore

Single collection named listings.

Page 12: Storage

Store listing images in Firebase Storage.

Page 13: UI Guidelines

Minimal, responsive, blue and white, rounded cards.

Page 14: Components

Navbar, Footer, SearchBar, ItemCard, ListingForm, ProtectedRoute, LoadingSpinner.

Page 15: Add Listing

Form validation, upload image, save Firestore document.

Page 16: Browse Listings

Responsive grid with cards.

Page 17: Search

Realtime filtering by item name.

Page 18: Item Details

Display full information and contact buttons.

Page 19: Security Rules

Authenticated users create listings. Owners edit/delete.

Page 20: Testing

Verify login, create listing, browse, search and image upload.

Page 21: Deployment

GitHub then Firebase Hosting.

Page 22: Git Workflow

Small commits after every phase.

Page 23: Coding Standards

Functional components, hooks, Tailwind only.

Page 24: AI Workflow

Use PROJECT.md and TASKS.md. Build one phase at a time.

Page 25: Debugging

Fix one error before adding features.

Page 26: Viva Notes

Explain React, Firebase, Firestore, CRUD and routing.

Page 27: Common Viva Questions

Why Firebase? Why React? Why NoSQL?

Page 28: Future Improvements

Payments, chat, ratings and notifications.

Page 29: Submission Checklist

Working app, GitHub repo, deployment, screenshots.

Page 30: Master Prompt

Read docs, implement one phase, explain changes, stop after completion.

Page 31: Conclusion

The goal is a clean, working MVP suitable for evaluation, not a production startup.